Decentralized Healthcare and Insurance Record Management Using Ethereum Smart Contracts

Shivam Kumar Jaiswal
Reg. No. 23BKT0079
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

Shibam Banik
Reg. No. 23BKT0108
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

Blockchain Architecture Design — Case Study III

***Abstract*—Fragmented health information and manual insurance workflows make it difficult for patients, healthcare providers, and insurers to coordinate records and payments with a consistent audit trail. This paper analyzes a decentralized healthcare and insurance prototype implemented with Ethereum-compatible smart contracts and a web client. The reviewed repository contains four principal Solidity contracts: MedicalRecords, Insurance, MedicalToken, and PriceOracle. A React/TypeScript client connects through MetaMask and Web3, while a Node.js oracle service can update demonstration premium and claim prices. Medical-record payloads are serialized and encrypted in the browser with CryptoJS AES before being submitted to the record contract; contract state stores the encrypted string, provider address, timestamp, and payment status. Insurance logic accepts monthly premiums in an ERC-20-compatible token and allows claims against active coverage, subject to contract balance and record-status checks. This paper presents the architecture, workflows, data model, access controls, implementation choices, and a source-based evaluation of the prototype. It also identifies material limitations: the inspected repository does not establish end-to-end test success, the oracle is locally configured and partly hard-coded, authorization and claim validation need strengthening, and client-side secret handling is unsuitable for production. Consequently, the system is best understood as an educational proof of concept rather than a deployable healthcare platform. No unsupported throughput, cost, or clinical effectiveness claims are made.**

***Index Terms*—blockchain, electronic medical records, Ethereum, smart contracts, health insurance, access control.**

I. Introduction

Healthcare information is routinely created by multiple organizations, including clinics, diagnostic laboratories, hospitals, and insurers. When systems are isolated, record exchange can require manual reconciliation and repeated verification. Insurance claims may also involve several parties checking eligibility, treatment documentation, and payment status. These processes create opportunities for delay, inconsistent data, and disputes over the history of a transaction. Distributed ledgers offer a shared execution and audit layer in which authorized participants can coordinate state transitions without relying on a single database operator [1], [2].

Blockchain is not, by itself, a complete solution to medical-data governance. Public-chain data is replicated and difficult to erase; therefore, placing readable personal health information directly on-chain is inappropriate. A more defensible design keeps sensitive payloads encrypted or off-chain while using smart contracts for permissions, identifiers, status flags, and auditable business rules. Prior work such as MedRec explored blockchain-based medical-data access and permission management, while later reviews have emphasized interoperability, privacy, governance, scalability, and deployment challenges [1], [2].

This project explores the integration of two related workflows: (i) provider submission and retrieval of electronic medical records (EMRs), and (ii) insurance premium payment and claim settlement. The implementation is a local Ethereum Virtual Machine (EVM) prototype, not a production health network. It uses Solidity contracts, Hardhat development tooling, a React/Vite/TypeScript client, MetaMask, Web3.js, a token contract, and an oracle-style price source. The source code and README are available in the project repository [3].

The paper makes four contributions. First, it documents the implemented modules and the path by which they interact. Second, it formalizes the principal record and insurance workflows. Third, it evaluates the source-level security and integration assumptions, distinguishing implemented mechanisms from intended outcomes. Fourth, it proposes a validation plan and engineering improvements required before any real-world deployment. The central research question is: to what extent can a modular Ethereum dApp provide a traceable prototype for linking medical-record events with tokenized insurance workflows, and what security and operational gaps remain?

II. Background and Related Work

A. Blockchain for Healthcare

A blockchain maintains replicated state and orders transactions through a consensus protocol. Smart contracts encode deterministic state transitions that are executed by participating nodes. In healthcare, these properties can support shared audit trails, provenance checks, and coordination among organizations that do not fully trust one another. They do not guarantee that submitted information is medically correct, that

the person entering it is legitimate, or that an application is compliant with privacy law. Data quality and identity assurance remain governance problems outside consensus [1], [2].

Azaria et al. introduced MedRec, a prototype using blockchain to manage permissions and references to medical information while integrating with providers' existing storage systems [1]. Kuo, Kim, and Ohno-Machado surveyed distributed-ledger applications in biomedical and healthcare settings, identifying potential benefits alongside limitations such as privacy, interoperability, performance, and adoption [2]. The present project follows the general idea of a shared ledger, but it differs in emphasis: its smart contracts also implement a simplified monthly insurance-premium and claim workflow, supported by a fungible utility token.

B. Ethereum and Smart Contracts

Ethereum smart contracts are programs that persist state and expose callable functions on the EVM [4]. Transactions can update state, emit events, and transfer assets; view calls read state without submitting a state-changing transaction. Gas meters computation and storage, which creates a practical reason to keep large documents off-chain. Solidity is the principal contract language used by this repository [5]. The EVM's deterministic execution makes it suitable for enforcing rules such as valid month ranges, token balances, and role checks, but contract correctness depends on implementation and testing.

C. Tokenized Payments

The project's MedicalToken contract builds on OpenZeppelin's ERC-20 implementation and adds owner-controlled minting. ERC-20 defines common operations such as balance queries, transfers, allowances, and delegated transferFrom calls [6], [7]. A payer approves the insurance contract to spend tokens before premium collection. Tokenization makes payment state programmable in the prototype; it does not establish that the token represents fiat currency, a legally binding insurance obligation, or a real-world reserve.

D. Design Positioning

The system combines familiar building blocks rather than proposing a new consensus algorithm or cryptographic primitive. Its intended contribution is an integrated application workflow. The appropriate evaluation criteria are therefore functional traceability, access-control correctness, transaction consistency, confidentiality boundaries, and reproducibility. Novelty should be claimed conservatively: the implementation is an educational integration of established Ethereum components, and no comparative experiment in the repository demonstrates superiority over a conventional database or insurer platform.

TABLE I
PROJECT COMPONENTS AND RESPONSIBILITIES

Component	Observed responsibility
MedicalRecords.sol	Provider authorization; encrypted record string; timestamp; payment flag.
Insurance.sol	Premium collection, active-coverage mapping, claim checks and token payout.
MedicalToken.sol	ERC-20 token with owner-controlled mint function and permit extension.
PriceOracle.sol	Owner-managed premium and claim values used by insurance logic.
React client	Wallet connection, forms, record retrieval, insurance status and claims.
Node.js oracle	Local Web3 connection and scripted price updates.

III. SYSTEM REQUIREMENTS AND ARCHITECTURE

A. Functional Requirements

The reviewed implementation supports a small set of demonstrable functions: connect a wallet; submit a medical record through a provider-oriented form; encrypt the serialized record in the browser; store the encrypted payload in a contract; retrieve and decrypt records through a repository class; authorize or remove providers through an administrator account; check whether coverage is active for a patient identifier and month; pay a premium using MedicalToken; and submit an insurance claim that transfers tokens to a provider address. The UI also displays a gas-price estimate when requested. These functions form a prototype workflow, not a complete claims-adjudication process.

B. Nonfunctional Requirements

The architecture aims for auditability, deterministic rule execution, modularity, and local reproducibility. It also has important nonfunctional requirements that are not fully met: confidentiality of patient data, robust identity binding, resistance to replay or duplicate claims, safe secret management, reliable oracle operation, accessibility, fault recovery, and scalability. These should be treated as acceptance criteria for future development rather than assumed properties.

C. Layered Architecture

The frontend is a Vite application built with React and TypeScript. Chakra UI components provide forms and layout, while React Hook Form handles form state. The Web3 context uses MetaMask SDK integration and connects the client to a local RPC endpoint at 127.0.0.1:8545. The contract-interaction module instantiates Web3 contract objects from generated ABI files and hard-coded local deployment addresses. The Solidity layer contains separate contracts for medical records, insurance, the MedicalToken ERC-20 token, and a price oracle. The Node.js oracle service uses Web3.js and Axios to connect to the local chain and issue owner-authorized price updates.

The repository therefore has a hybrid architecture: user interaction and encryption occur in the browser; authorization, record metadata, coverage flags, and token transfers are

handled on-chain; and oracle update automation runs off-chain. The oracle is not a decentralized oracle network. The checked-in service targets a local node, contains a configured contract address and a development account/private key, and must not be reused as-is outside a disposable test chain (see Fig. 1).

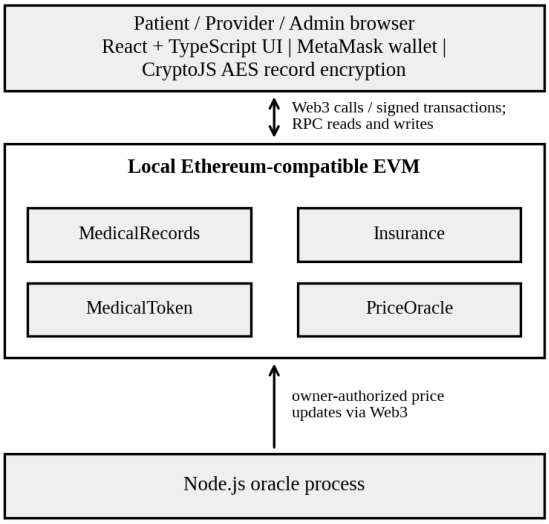


Fig. 1. Logical architecture of the prototype.

D. Deployment Model

The README describes separate startup commands for the client, oracle, and local chain/contracts. The frontend uses environment variables for blockchain URL, contract addresses, and an encryption secret in the medical-record repository, while the Ethereum client helper also contains local addresses. The contracts are compiled with Solidity 0.8.x-compatible syntax and configured through Hardhat. The local chain is useful for demonstrations because accounts and transactions can be reset; it does not provide production network resilience, identity assurance, or real insurance settlement.

IV. Smart Contract Design

A. MedicalRecords Contract

MedicalRecords.sol defines a Record structure containing encryptedData, provider, timestamp, and isPaid. Records are grouped in a mapping keyed by a 64-bit patient identifier (named NIK in the code). A separate mapping tracks authorized provider addresses. The deployer becomes admin and can authorize or remove providers. The admin also configures an allowedContract address. Only authorized providers may append records or retrieve record arrays/counts; only the configured contract may update the payment status. Events expose record creation, provider authorization/removal, and payment-status changes.

This separation creates a useful role boundary: providers can add clinical payloads, the administrator manages provider eligibility, and the insurance contract can mark a record paid. However, getRecords restricts reads by provider authorization, not by patient ownership or patient consent. The code does not implement patient-specific consent revocation, insurer-scoped access, key rotation, or a cryptographically verifiable relationship between a provider's identity and a licensed institution. The 64-bit identifier is a direct key and must not be treated as anonymization.

B. Insurance Contract

Insurance.sol stores references to the token, oracle, and medical-record contracts. Its payPremium function obtains the current premium from the oracle, validates that the month is between 1 and 12 and that the year is nonzero, checks the caller's token balance and allowance, marks coverage active for the supplied NIK/year/month, emits PremiumPaid, and transfers tokens from the caller to the contract. The active-coverage mapping is keyed by patient identifier, year, and month.

The claim function obtains the claim amount from the oracle, validates the date fields, requires active coverage, checks the contract's token balance, asks MedicalRecords to mark a selected record as paid, emits a Claimed event, and transfers tokens to the provider address. The flow couples record status with token payout. Yet the function does not independently prove that the selected record exists, belongs to the caller, falls within the claimed month, or is linked to the specified provider. It also does not visibly prevent a second claim for the same active month by checking the existing isPaid flag before payout. These are substantive business-logic gaps, not merely documentation issues.

C. MedicalToken and PriceOracle

MedicalToken.sol inherits OpenZeppelin ERC20, ERC20Permit, and Ownable. It defines the token name as MedicalToken, symbol MTK, and exposes owner-only minting. This is suitable for local demonstrations, but unrestricted owner minting means the owner can increase supply without a cap. PriceOracle.sol stores premiumPrice and claimPrice and allows only its owner to update them. Although the source includes an oracle service that updates values, the contract itself is a simple owner-controlled data store rather than an independent external-data verification system.

D. Events and Auditability

Events such as PremiumPaid, Claimed, ProviderAuthorized, and RecordAdded give an observer a transaction-linked history of important state changes. Events improve traceability but are not a complete audit framework: event schemas must be consistent, relevant identifiers must be indexed carefully, and an application must verify transaction receipts and event values before showing success. The current frontend sometimes displays an informational success message immediately after initiating an asynchronous claim call; a production UI should wait for the transaction receipt and final status.

V. Medical Data Confidentiality and Access Control

A. Client-Side Encryption

MedicalRecordsRepository.ts serializes a MedicalRecord object as JSON and encrypts the string using CryptoJS AES with a secret supplied through VITE_SECRET_KEY. The resulting ciphertext is passed to addRecord; the retrieval path calls the contract, decrypts encryptedData using the same secret, and reconstructs the record. The on-chain payload is therefore intended to be ciphertext rather than readable diagnosis or treatment text. This is a better privacy boundary than placing plaintext clinical data directly on a public ledger.

The protection is only as strong as key generation, key distribution, storage, and runtime exposure. In Vite, variables prefixed with VITE_ are bundled for client-side use; a value embedded in browser code is not a server-side secret. Anyone able to inspect the built application may recover the key and decrypt records. The frontend also logs environment information and clinical form data to the console in several places. Therefore, the implementation's AES transformation must not be described as end-to-end privacy against application operators or unauthorized users who can obtain the client secret. No key-management service, per-patient keys, rotation mechanism, hardware-backed key protection, or independent security audit is evidenced in the inspected files.

B. On-Chain Data and Metadata

Even when ciphertext is stored, the chain exposes metadata such as provider address, timestamps, record counts, event history, and the patient identifier used as a key. Because transactions and contract storage are replicated, data written to a public network can be difficult or impossible to erase. A production design should avoid placing direct identifiers and ciphertext on a public chain unless the threat model and legal basis explicitly justify it. A safer architecture typically stores encrypted records in a controlled off-chain repository and anchors a hash or content commitment on-chain; access to the underlying object is then mediated by authenticated services.

C. Access-Control Findings

The onlyAdmin, onlyAuthorizedProvider, onlyAllowedContract, and onlyOwner modifiers are useful initial controls. They constrain selected state-changing functions by Ethereum address. However, addresses are pseudonymous keys, not proof of professional credentials. The system does not include a verifiable credential mechanism for clinicians or insurers, nor does it bind a wallet to a real patient through a trusted enrollment workflow. The UI's isAdmin state is a client-side toggle and must not be mistaken for contract authorization. Security decisions must be enforced by contracts and authenticated services, never solely by hiding or showing UI elements.

D. Threat Model

Relevant adversaries include a malicious patient attempting duplicate claims; a provider submitting false or mismatched records; an unauthorized wallet attempting reads or writes; a compromised browser or exposed encryption key; a malicious or compromised oracle operator; and an attacker exploiting incorrect contract addresses or RPC configuration. Potential failures include replayed claims, identifier substitution, unauthorized record disclosure, owner-key compromise, token inflation, denial of service through an unavailable local node, and inconsistent UI state. The current implementation provides partial controls but does not demonstrate mitigation of all these cases.

E. Privacy and Compliance Boundary

The prototype should use synthetic records only. It is not evidence of compliance with HIPAA, GDPR, India's Digital Personal Data Protection framework, or any healthcare regulation. Compliance depends on jurisdiction, deployment model, controller/processor roles, consent, retention, breach response, access logging, and operational policies in addition to cryptographic controls. Before clinical use, a privacy impact assessment and qualified legal/security review would be mandatory.

VI. End-to-End Workflows

A. Provider Record Submission

The provider connects a MetaMask account to the local EVM. The client form collects a checkup date, provider label, NIK, name, diagnosis, and treatment. MedicalRecordsRepository serializes these fields, encrypts the JSON using the configured secret, and sends a transaction to MedicalRecords.addRecord. The contract checks that the sender is an authorized provider, appends a record with the current block timestamp and an initial isPaid=false flag, and emits RecordAdded. The transaction receipt is used by the repository to obtain a record index. A successful submission is thus represented by both persisted contract state and an event.

B. Premium Payment

The payer enters a patient identifier and submits an insurance form. The client reads the premium amount and token balance, approves the Insurance contract to spend tokens, and invokes payPremium with a month, year, and NIK. The contract reads the price from PriceOracle, checks the month/year and balance/allowance, records coverage as active, emits PremiumPaid, and transfers the premium to the insurance contract. The process depends on the token's allowance model and the oracle's configured value. In a production implementation, the UI should show the exact token units and require explicit confirmation of the month, amount, recipient contract, and network.

C. Coverage Check and Claim

The coverage-check UI calls isActive for a NIK and period. To claim, the user fetches medical records, selects a record, and triggers Insurance.claim with the period, provider, NIK, and record index. The contract checks active coverage and contract

token balance, calls updateRecordPaymentStatus, emits Claimed, and transfers tokens to the provider. This is the intended automation point: deterministic conditions replace some manual coordination. Nevertheless, the current contract does not fully validate the selected record's ownership, date, provider association, or prior payment status. The workflow must therefore be described as a basic claim settlement demonstration, not a complete fraud-resistant adjudication system.

D. Oracle Price Update

The oracle process connects to a local JSON-RPC endpoint using Web3.js and contains an ABI and address for a price-oracle contract. It signs updates with a configured development account. PriceOracle restricts updates to its owner. This arrangement is useful for simulating changing premium and claim amounts, but it has a single trusted updater and no verifiable data provenance. A robust oracle would authenticate source data, apply validation and bounds, record source/time metadata, protect signing keys, and provide fallback behavior when data is unavailable.

E. Transaction State and User Feedback

A blockchain transaction is not equivalent to a completed operation merely because a button was clicked. The user interface should distinguish submitted, pending, confirmed, failed, and reverted states. The current source includes asynchronous calls, event listeners, and console output, but transaction completion is not consistently reflected in the UI. The recommended interaction sequence is: validate form; request wallet signature; submit transaction; await receipt; decode relevant event; refresh contract state; then display success. If a transaction reverts, show a safe explanation without exposing secrets or sensitive record contents.

F. Deployment Reproducibility

The README documents starting the local chain, deploying contracts, running the client and oracle, importing a development wallet, and opening the Vite application at localhost:5173 [3]. Reproducibility requires the deployed addresses and ABIs to match the current chain. The repository contains hard-coded local contract addresses in the client, so a fresh deployment can invalidate them unless the address-generation step updates configuration. A deployment manifest and automated smoke test should be introduced to ensure all modules reference the same contract instances.

VII. Implementation and Repository Review

A. Technology Stack

The contract workspace uses Hardhat, ethers.js, TypeScript deployment modules, Solidity 0.8.27 for the principal contracts, and OpenZeppelin Contracts. The frontend uses React 18, TypeScript, Vite, Chakra UI, MetaMask SDK React integration, Web3.js, and CryptoJS. The oracle package is a Node.js ES module using Web3.js and Axios. This separation is maintainable in principle: the frontend handles interaction, Solidity controls ledger state, and the oracle process updates configurable prices.

B. Contract Deployment and Address Coupling

The repository includes Hardhat Ignition modules for MedicalRecords, MedicalToken, PriceOracle, and insurance-related deployment. The client's src/eth/app.ts defines fixed local addresses and loads contract ABIs from src/abi. This creates a coupling between deployment order, network state, generated artifacts, and frontend configuration. An address mismatch can make a valid UI call target an empty account or a different contract. Deployment should output a versioned JSON manifest containing chain ID, contract addresses, compiler version, artifact hashes, and deployment transaction hashes; the frontend should consume that manifest rather than duplicate addresses in source.

C. Test Coverage Observed

The repository contains a default Hardhat Lock test, a MedicalRecords test suite, and a MedicalToken test. The MedicalRecords tests check provider authorization, adding and retrieving a record, setting the allowed contract, and an unauthorized call. The token test checks the deployed token name. The inspected test inventory does not include a dedicated Insurance test suite, and there is no evidence in this review that tests were executed successfully. These tests should not be interpreted as full system validation.

D. Static Review Findings

The code review identifies several correctness and security issues to prioritize. First, Insurance.claim should verify that the referenced record exists, is unpaid, belongs to the requested patient and month, and names the same provider that will receive the transfer. Second, premium payment can mark coverage active based on a caller-supplied NIK without proving that the payer is entitled to insure that identifier. Third, repeated claims may be possible because the claim path does not visibly reject an already-paid record before transfer. Fourth, PriceOracle.setClaimPrice emits PremiumUpdated rather than ClaimUpdated, which can confuse monitoring. Fifth, Insurance stores premiumAmount and claimAmount but reads current values from the oracle; the local admin setters therefore do not drive the active price path. Sixth, the frontend and oracle contain local configuration and a development private key pattern that must never be used with real assets.

E. Client Integration Risks

The UI includes a client-side admin toggle; this is not a secure role system. Some record retrieval paths use a repository instantiated with a VITE-prefixed secret. The claim component initiates asynchronous actions and can show feedback before final transaction confirmation. Several date strings are split and cast to numbers in a way that is difficult to audit. The premium form reads a current-month value even though a month input is displayed, and token approval arithmetic should be reviewed

for the token's 18-decimal default and exact smallest-unit conventions [7]. These findings require functional testing and code correction before any claim of reliable operation.

F. Research Method

This paper uses a repository-based design and static implementation review: README and file-tree inspection, analysis of the four Solidity contracts, review of the frontend's record and claim components, inspection of Web3 configuration, and review of the available test files. It is not a controlled benchmark, formal verification, penetration test, or legal compliance audit. No gas, latency, throughput, or user-study results are reported because no reproducible measurements were available in the repository review.

VIII. Evaluation Plan and Discussion

A. Functional Test Matrix

A research-grade evaluation should test both expected and adversarial behavior. Table II defines the minimum tests required; it is a proposed plan, not a report of completed runs.

TABLE II
Required Validation Scenarios

Area	Test case	Expected result
Record access	Unauthorized wallet calls addRecord/getRecords.	Revert; no record disclosure or state change.
Provider lifecycle	Admin authorizes, removes, then provider submits.	Write allowed only while authorized.
Premium	Insufficient balance or allowance.	Revert; coverage remains inactive.
Period validation	Month 0 or 13; year 0.	Revert with no state change.
Claim eligibility	Claim while coverage inactive.	Revert; no token transfer.
Duplicate claim	Submit same record claim twice.	Second claim rejected.
Record binding	Wrong NIK, provider, or period.	Claim rejected before payout.
Oracle access	Non-owner changes price.	Revert; stored price unchanged.
Integration	Fresh deployment with generated manifest.	Client calls correct contract addresses.
Privacy	Inspect browser bundle and console logs.	No recoverable master secret or plaintext PHI.

B. Quantitative Metrics

After correctness tests pass, performance can be measured using a local chain and a fixed test dataset. Suggested metrics are transaction gas used for record insertion, premium payment, and claim settlement; median and 95th-percentile confirmation time; success/revert rate under controlled inputs; and storage growth per record. Results should include compiler version, optimizer settings, chain configuration, hardware, number of repetitions, and whether time includes wallet interaction. Since local Hardhat timing is not representative of a public network, public-chain estimates should be reported separately.

C. Security Evaluation

A contract audit should focus on authorization, reentrancy and external calls, token transfer assumptions, integer/unit correctness, duplicate settlement, identifier binding, owner privileges, and oracle trust. The use of standard OpenZeppelin ERC-20 components reduces the need to implement token primitives manually, but does not make the application contract automatically safe [6]. Static analysis tools, fuzzing, invariant testing, and manual review should complement ordinary unit tests. High-value invariants include: a record can be paid at most once; an inactive policy cannot settle a claim; no claim pays more than the configured claim amount; and a failed transaction leaves both record and token state unchanged.

D. Discussion

The prototype demonstrates a coherent architectural direction: record payloads are encrypted before storage, role checks are enforced by the record contract, premiums and claims are represented as token transfers, and events provide an audit trail. The modular contract design makes individual responsibilities easier to inspect. However, the current system's practical trust model remains centralized in several places: an admin authorizes providers, an owner controls token minting and oracle prices, and the browser holds the encryption secret. Claim eligibility relies heavily on caller-supplied identifiers. The benefits are therefore traceability and programmable workflow in a local demonstration, not complete decentralization, privacy, or fraud elimination.

E. Validity Threats

This evaluation is limited by the snapshot of source code available in the public repository at review time, the lack of executed integration tests, and the absence of deployment logs or performance data. Some frontend and oracle behavior depends on environment variables and a local chain. Consequently, conclusions about architecture and source-level behavior are grounded in the inspected files, while claims about runtime reliability remain unverified. Future versions should attach automated test reports and deployment artifacts to make results reproducible.

IX. Limitations, Ethical Considerations, and Future Work

A. Current Limitations

The implementation is an educational prototype and should not be deployed with real patient data or real funds. The local RPC endpoint, fixed contract addresses, and development wallet assumptions limit portability. The encryption secret is delivered to the browser, which defeats strong separation between application code and secret material. The oracle is owner-controlled and uses a local service rather than a decentralized or independently attested source. The claims contract lacks robust validation of record existence, date/provider consistency, patient entitlement, and duplicate payment. The repository does not demonstrate comprehensive

insurance tests, performance benchmarks, formal verification, or a production deployment.

The system also leaves identity and governance unresolved. An Ethereum address is not a medical license, a patient identity, or proof of policy ownership. A real system would need enrollment and recovery processes, audited role assignment, revocation, consent semantics, dispute handling, retention rules, and legal agreements among providers and insurers. Medical data can remain sensitive even after encryption because metadata and access patterns can reveal relationships and activity.

B. Recommended Engineering Changes

1) Move medical payloads to encrypted off-chain storage and anchor only a cryptographic digest or content address on-chain.
2) Replace a shared frontend secret with authenticated key management, per-patient or per-record data keys, secure key wrapping, rotation, and revocation.
3) Enforce record-to-claim binding in the contract, including existence, provider, date, patient, and unpaid status; add a claim identifier and replay protection.
4) Bind coverage to a verified policyholder relationship rather than accepting a free-form patient identifier.
5) Replace ad hoc owner price updates with an audited oracle interface, input validation, freshness limits, and emergency fallback.
6) Use SafeERC20 for robust token interactions and define explicit token units.
7) Generate and consume a deployment manifest.
8) Await receipts and display transaction state correctly.
9) Remove console logging of personal data and all browser-visible secrets.
10) Add CI for compile, lint, unit tests, integration tests, and security analysis.

C. Future Research Directions

Future work should compare the proposed workflow with a conventional database-backed claims application using the same synthetic workload. A useful experiment would measure transaction cost, latency, and audit reconstruction time while varying record size, number of providers, and claim rate. Privacy research could compare on-chain hashes with off-chain encrypted storage, including the implications of key revocation and metadata leakage. Access-control research could add verifiable credentials for providers and consent tokens for patients. Finally, a permissioned deployment or layer-2 network could be evaluated where governance, transaction throughput, and cost are more appropriate for a consortium of healthcare organizations.

D. Responsible Use

All demonstrations should use synthetic identities and invented clinical details. The test private key described by local development tooling is public test material and must never hold valuable assets. Real-world deployment would require threat modeling, independent smart-contract review, penetration testing, secure infrastructure, incident response, and legal review in every applicable jurisdiction. A blockchain's immutability is not an excuse to retain data without a lawful purpose; it is a reason to minimize on-chain personal information from the outset.

E. Reproducibility Checklist

Before reporting the system as validated, publish the exact commit hash; dependency lockfiles; compiler and optimizer configuration; deployment manifest; test command and complete test output; seeded synthetic dataset; gas-measurement script; and threat-model assumptions. Report failures as well as passing cases. Separate local-chain observations from public-network estimates and avoid claims of production security without independent evidence.

X. Conclusion

This paper presented a source-based analysis of a decentralized healthcare and insurance prototype that links encrypted medical-record storage with monthly tokenized insurance premiums and provider-directed claim payments. The implementation combines MedicalRecords, Insurance, MedicalToken, and PriceOracle contracts with a React/TypeScript frontend, MetaMask/Web3 integration, and a Node.js oracle process. It demonstrates how blockchain state and smart-contract events can provide a shared transaction history for selected healthcare and insurance operations.

The repository also illustrates why a functional prototype should not be confused with a production system. Encryption occurs in the browser using a shared environment-provided secret; identity and consent are incomplete; oracle updates rely on an owner; the insurance claim path requires stronger record validation and duplicate-settlement protection; and the available test suite does not establish end-to-end correctness. The system therefore offers a useful educational basis for studying decentralized application architecture, but it does not yet prove confidentiality, fraud prevention, regulatory compliance, scalability, or production readiness.

The next milestone should be a correctness-first hardening phase: implement strict record-to-claim invariants, remove browser-visible secrets, externalize encrypted clinical data, formalize identity and consent, improve oracle trust and deployment configuration, and add comprehensive tests. Only after those changes should the project be evaluated quantitatively against a conventional baseline. This conservative path preserves the prototype's value while making its technical and ethical boundaries explicit.

Acknowledgment

The authors acknowledge the open-source Ethereum, Solidity, Hardhat, OpenZeppelin, React, Web3.js, and MetaMask ecosystems on which this educational prototype builds. The repository analyzed in this paper is available at:

https://github.com/ShivamKJ1/Decentralized-Healthcare-Insurance-Ethereum-Smart-Contracts.

REFERENCES

[1] A. Azaria, A. Ekblaw, T. Vieira, and A. Lippman, “MedRec: Using blockchain for medical data access and permission management,” in *Proc. 2nd Int. Conf. Open Big Data (OBD)*, Vienna, Austria, 2016, pp. 25–30, doi: 10.1109/OBD.2016.11.

[2] T.-T. Kuo, H.-E. Kim, and L. Ohno-Machado, “Blockchain distributed ledger technologies for biomedical and health care applications,” *J. Amer. Med. Inform. Assoc.*, vol. 24, no. 6, pp. 1211–1220, 2017, doi: 10.1093/jamia/ocx068.

[3] S. K. Jaiswal, “Decentralized healthcare insurance Ethereum smart contracts,” GitHub repository, 2026. Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/ShivamKJ1/Decentralized-Healthcare-Insurance-Ethereum-Smart-Contracts

[4] Ethereum Foundation, “Ethereum Virtual Machine (EVM),” Ethereum.org. Accessed: Oct. 13, 2026. [Online]. Available: https://ethereum.org/developers/tutorials/yellow-paper-evm/

[5] Solidity Team, “Solidity documentation,” Solidity. Accessed: Oct. 13, 2026. [Online]. Available: https://docs.solidity.org/en/latest/

[6] OpenZeppelin, “OpenZeppelin Contracts documentation,” OpenZeppelin. Accessed: Oct. 13, 2026. [Online]. Available: https://docs.openzeppelin.com/contracts

[7] F. Vogelsteller and V. Buterin, “EIP-20: Token standard,” Ethereum Improvement Proposals, no. 20, Nov. 2015. Accessed: Oct. 13, 2026. [Online]. Available: https://eips.ethereum.org/EIPS/eip-20